

Text File Handling in Java

12 June 2026 18:16

Text File handling is a process of storing and retrieving data from a text file(.txt) using Java programs.

A text file stores data in a human-readable format.

Examples:

student.txt
marks.txt
poem.txt
story.txt

Java provides several classes for file handling:

- File
- FileWriter
- FileReader
- BufferedWriter
- BufferedReader
- PrintWriter
- Scanner

All these classes belongs to the java.io package (except Scanner which belongs to java.util package)

Why Use Files?

Variable store data temporarily in memory. When the program terminates data is lost.

Files provide:

- Permanent storage
- Data sharing
- Data backup
- Record maintenance

File Class

The File class is used to create, delete and manage files.
(We already studied about it.)

Writing Data to a Text File

Using FileWriter class

Class FileWriter

```
java.lang.Object
  java.io.Writer
    java.io.OutputStreamWriter
      java.io.FileWriter
```

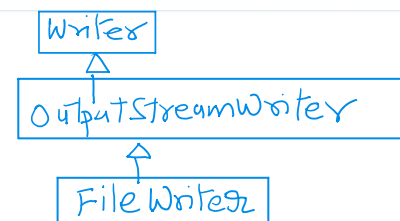
All Implemented Interfaces:

Closeable, Flushable, Appendable, AutoCloseable

```
public class FileWriter
extends OutputStreamWriter
```

Writes text to character files using a default buffer size. Encoding from characters to bytes uses either a specified charset or the default charset.

Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one FileWriter (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved



Class FileWriter

```
java.lang.Object
  java.io.Writer
    java.io.OutputStreamWriter
      java.io.FileWriter
```

All Implemented Interfaces:

Closeable, Flushable, Appendable, AutoCloseable

```
public class FileWriter
extends OutputStreamWriter
```

Writes text to character files using a default buffer size. Encoding from characters to bytes uses either a specified `charset` or the default `charset`.

Whether or not a file is available or may be created depends upon the underlying platform. Some platforms, in particular, allow a file to be opened for writing by only one `FileWriter` (or other file-writing object) at a time. In such situations the constructors in this class will fail if the file involved is already open.

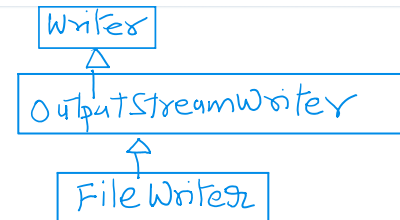
The `FileWriter` is meant for writing streams of characters. For writing streams of raw bytes, consider using a `FileOutputStream`.

Since:

1.1

See Also:

`OutputStreamWriter`, `FileOutputStream`, `Charset.defaultCharset()`



Constructors

Constructor	Description
✓ <code>FileWriter(File file)</code>	Constructs a <code>FileWriter</code> given the <code>File</code> to write, using the default <code>charset</code> .
<code>FileWriter(FileDescriptor fd)</code>	Constructs a <code>FileWriter</code> given a file descriptor, using the default <code>charset</code> .
✓ <code>FileWriter(File file, boolean append)</code>	Constructs a <code>FileWriter</code> given the <code>File</code> to write and a boolean indicating whether to append the data written, using the default <code>charset</code> .
✓ <code>FileWriter(File file, Charset charset)</code>	Constructs a <code>FileWriter</code> given the <code>File</code> to write and <code>charset</code> .
<code>FileWriter(File file, Charset charset, boolean append)</code>	Constructs a <code>FileWriter</code> given the <code>File</code> to write, <code>charset</code> and a boolean indicating whether to append the data written.
✓ <code>FileWriter(String fileName)</code>	Constructs a <code>FileWriter</code> given a file name, using the default <code>charset</code> .
✓ <code>FileWriter(String fileName, boolean append)</code>	Constructs a <code>FileWriter</code> given a file name and a <u>boolean indicating whether to append the data written</u> , using the default <code>charset</code> .
<code>FileWriter(String fileName, Charset charset)</code>	Constructs a <code>FileWriter</code> given a file name and <code>charset</code> .
<code>FileWriter(String fileName, Charset charset, boolean append)</code>	Constructs a <code>FileWriter</code> given a file name, <code>charset</code> and a boolean indicating whether to append the data written.

```
package FileHandling;
import java.io.*;
import java.util.*;
```

```
/**
 * Write a description of class WriteFile here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class WriteFile
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String fileName, path = "./DataFiles/";
        System.out.print("\nEnter file name with .txt extension: ");
        fileName = sc.nextLine();
    }
}
```

```

try{
    FileWriter fw = new FileWriter(path+fileName, true);
    System.out.print("\nEnter Roll number: ");
    int rollNum = sc.nextInt();
    sc.nextLine(); //clears the keyboard buffer
    System.out.print("Enter name: ");
    String name = sc.nextLine();
    System.out.print("Enter marks: ");
    float marks = sc.nextFloat();
    fw.write("Roll no: " + rollNum+"\n");
    fw.write("Name: " + name + "\n");
    fw.write("Marks: " + marks + "\n");

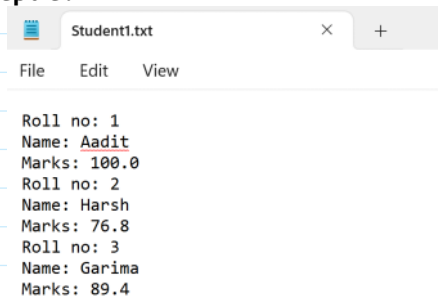
    fw.close();
    System.out.print("\nData Written");
}catch(IOException ioe){
    System.err.print("\nFile is not created");
}

}
}

```

↑ append mode is on

Output:



```

Student1.txt
File Edit View

Roll no: 1
Name: Aadit
Marks: 100.0
Roll no: 2
Name: Harsh
Marks: 76.8
Roll no: 3
Name: Garima
Marks: 89.4

```

Reading Data from a Text File Using FileReader

```

package FileHandling;
import java.util.*;
import java.io.*;

/**
 * Write a description of class ReadFile here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class ReadFile
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String path="./DataFiles/", fileName;
        System.out.print("\nEnter file name as .txt extension: ");
        fileName = sc.nextLine();
        try{

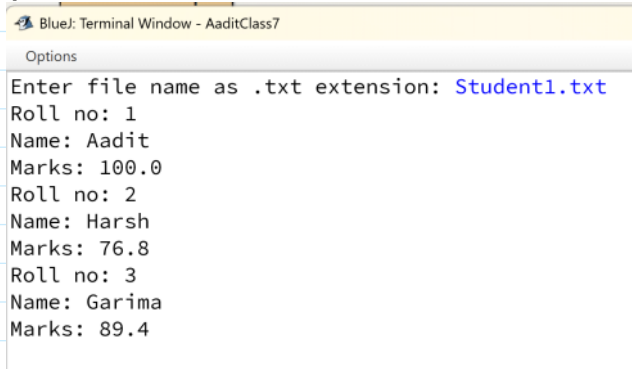
```

```

        FileReader fr = new FileReader(path+fileName);
        int ch;
        while((ch = fr.read())!= -1){
            System.out.print((char)ch);
        }
        fr.close();
    }catch(FileNotFoundException fnfe){
        System.err.print("\nFile nahi mila.");
    }
    catch(IOException ioe){
        System.err.print("\nNetwork Error!");
    }
}
}

```

Output:



```

BlueJ: Terminal Window - AaditClass7
Options
Enter file name as .txt extension: Student1.txt
Roll no: 1
Name: Aadit
Marks: 100.0
Roll no: 2
Name: Harsh
Marks: 76.8
Roll no: 3
Name: Garima
Marks: 89.4

```

Using BufferedWriter

BufferedWriter improves writing performance.

```

package FileHandling;
import java.util.*;
import java.io.*;

/**
 * Write a description of class UsingBufferedWriter here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UsingBufferedWriter
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String fileName, path = "./DataFiles/";
        System.out.print("\nEnter file name with .txt extension: ");
        fileName = sc.nextLine();
        try{
            FileWriter fw = new FileWriter(path+fileName, true);
            BufferedWriter bw = new BufferedWriter(fw);
            System.out.print("\nEnter Roll number: ");
            int rollNum = sc.nextInt();
            sc.nextLine(); //clears the keyboard buffer
            System.out.print("Enter name: ");

```

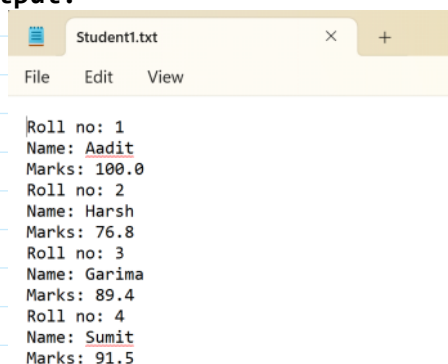
```

        String name = sc.nextLine();
        System.out.print("Enter marks: ");
        float marks = sc.nextFloat();
        bw.write("Roll no: " + rollNum);
        bw.newLine();
        bw.write("Name: " + name);
        bw.newLine();
        bw.write("Marks: " + marks);
        bw.newLine();

        bw.close();
        System.out.print("\nData Written");
    } catch (IOException ioe) {
        System.err.print("\nFile is not created");
    }
}
}
}

```

Output:



```

Student1.txt
File Edit View

Roll no: 1
Name: Aadit
Marks: 100.0
Roll no: 2
Name: Harsh
Marks: 76.8
Roll no: 3
Name: Garima
Marks: 89.4
Roll no: 4
Name: Sumit
Marks: 91.5

```

→ write mode → it creates a new file & we write data in it. In write mode, data overwrites.

→ append mode → In append mode data will be added at the end of existing file. If file is not existing then file is created and data is added.

→ read mode → we read the data only from existing file.

Using BufferedReader

BufferedReader reads line-by-line.

```

package FileHandling;
import java.util.*;
import java.io.*;

/**
 * Write a description of class UsingBufferedReader here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class UsingBufferedReader
{
    public static void main(String[] args){
        Scanner sc = new Scanner(System.in);
        String fileName, path = "./DataFiles/";
        System.out.print("\nEnter file name with .txt extension: ");
        fileName = sc.nextLine();
        try{
            FileReader fr = new FileReader(path+fileName);
            BufferedReader br = new BufferedReader(fr);

```

```

        String line;
        while((line = br.readLine())!= null){
            System.out.print("\n"+line);
        }
        br.close();
        sc.close();
    }catch(FileNotFoundException fnfe){
        System.err.print("\nFile not found on the specified path!");
    }catch(IOException ioe){
        System.err.print("\nFile is not accessible!");
    }
}
}

```

Output:

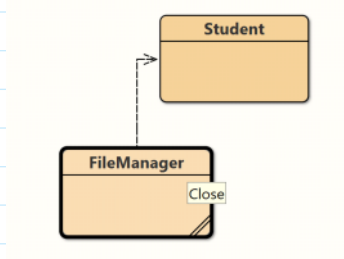
```

BlueJ: Terminal Window - AaditClass7
Options
Enter file name with .txt extension: Student1.txt

Roll no: 1
Name: Aadit
Marks: 100.0
Roll no: 2
Name: Harsh
Marks: 76.8
Roll no: 3
Name: Garima
Marks: 89.4
Roll no: 4
Name: Sumit
Marks: 91.5

```

Object-Oriented Approach Using Classes



CRUD Operations Using Text Files

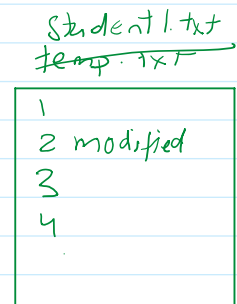
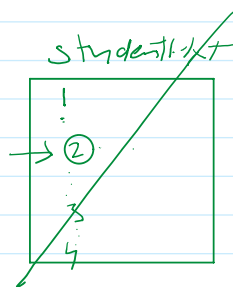
CRUD stands for:

C=> Create
R=> Read
U=> Update
D=> Delete

Update:

Steps:

- 1) Read original file
- 2) Copy records into temporary File
- 3) Modify desired record and place it into temporary file.
- 4) Delete the original file.
- 5) Rename the temporary file with name of original file.



if (n U-num == row) {
 marks ← take input
 ...
 }

- 3) Modify desired record and place it into temporary file.
- 4) Delete the original file.
- 5) Rename the temporary file with name of original file.

*if (roll-num == roll)?
marks ← take input
send old roll-num,
name, & new marks
to temp file.*

Delete:

Steps:

- 1) Read original file
- 2) Skip records to delete
- 3) Send rest of the record into temporary file.
- 4) Delete the original file.
- 5) Rename the temporary file with name of original file.

Try-with-resource(Recommended technique)

Automatically closes the file.

```
package FileHandling;
import java.util.*;
import java.io.*;
```

```
/**
 * Write a description of class FileManager here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class FileManager
{
    private static Scanner sc = new Scanner(System.in);
    public static void saveStudent(Student stu[]){
        String fileName, path = "./DataFiles/";
        System.out.print("\nEnter file name with .txt extension: ");
        fileName = sc.nextLine();

        try(FileWriter fw = new FileWriter(path+fileName, true);
            BufferedWriter bw = new BufferedWriter(fw)
            ){
            int i;
            for(i = 0; i < stu.length; i++){
                bw.write("Roll no:"+Integer.toString(stu[i].getRoll()));
                bw.newLine();
                bw.write("Name: "+stu[i].getName());
                bw.newLine();
                bw.write("Marks: "+Double.toString(stu[i].getMarks()));
                bw.newLine();
            }
            System.out.print("\n" + stu.length + " records saved in the
file.");

        }catch(IOException io){
            System.out.print("\nFile is not created");
        }
    }

    public static void main(String[] args){
        int howMany;
```

```

        System.out.print("\fEnter number of Students: ");
        howMany = sc.nextInt();
        Student st[] = new Student[howMany];
        int i;

        for(i = 0; i < howMany; i++){
            System.out.print("Enter Roll Number: ");
            int rn = sc.nextInt();
            sc.nextLine(); //clearing keyboard buffer.
            System.out.print("Enter name: ");
            String nm = sc.nextLine();
            System.out.print("Enter marks: ");
            double m = sc.nextDouble();
            sc.nextLine();//clearing keyboard buffer.
            st[i] = new Student(rn, nm, m); //creating objects
        }
        saveStudent(st);
    }
}

```

Home work

Try to complete Add, Display OR (create and read) features of the first 3 questions

Long Programming Questions

1. Student Record Management System

Create a Java program using classes and text file handling to maintain student records. The program should allow the user to:

- Add a student record
- Display all records
- Search a student by roll number
- Update a student's marks
- Delete a student record

Store all records in a text file named students.txt.

2. Employee Management System

Design an Employee class containing:

- Employee ID
- Name
- Department
- Salary

Write a menu-driven Java program that stores employee records in a text file and performs CRUD (Create, Read, Update, Delete) operations.

3. Library Management System

Create a Java application to manage book records using text files.

Each book should contain:

- Book ID
- Title
- Author
- Price

Implement features to:

- Add books
- Display books
- Search books by ID

- Update book details
- Delete books

Use `BufferedReader` and `BufferedWriter`.

4. Product Inventory System

Develop a Java program that stores product information in a text file.

Each product contains:

- Product ID
- Product Name
- Quantity
- Price

The system should:

- Add new products
- Display all products
- Search a product
- Update quantity
- Delete a product

Generate a stock report showing total inventory value.

5. Examination Result Processing System

Create a Java class `StudentResult` having:

- Roll Number
- Name
- Marks in 5 Subjects

Store records in a text file.

The program should:

- Calculate total marks
- Calculate percentage
- Assign grades
- Display topper details
- Display failed students

6. Contact Management System

Develop a Java program using text files to maintain contact information.

Each contact contains:

- Contact ID
- Name
- Mobile Number
- Email Address

Implement:

- Add Contact
- Search Contact
- Update Contact
- Delete Contact
- Display All Contacts

Use exception handling throughout the program.

7. Bank Account Management System

Create an `Account` class with:

- Account Number
- Customer Name
- Balance

Store account details in a text file.

Implement:

- Open Account
- Deposit Amount
- Withdraw Amount
- Check Balance
- Display All Accounts

Ensure proper validation and exception handling.

8. Attendance Management System

Design a Java application that stores attendance details in a text file.

Each record contains:

- Roll Number
- Student Name
- Attendance Percentage

The system should:

- Add attendance records
- Display attendance records
- Search by roll number
- Update attendance percentage
- Generate a list of students having attendance below 75%

9. Hotel Reservation System

Create a Java program to maintain room booking details using text files.

Store:

- Room Number
- Customer Name
- Number of Days
- Room Type

Operations:

- Book Room
- Cancel Booking
- Search Booking
- Update Booking
- Display All Bookings

Calculate total room charges automatically.

10. University Admission Management System

Design an object-oriented Java application using text file handling.

Create a class Applicant containing:

- Application Number
- Name
- Course Applied
- Entrance Marks

The program should:

- Add applicant details
- Display all applicants
- Search an applicant
- Update applicant information
- Delete applicant records
- Generate a merit list in descending order of marks

Use FileReader, FileWriter, BufferedReader, and BufferedWriter.

